

Transmission vidéo basse latence pour drone FPV

Travail de Bachelor

Non Confidentiel

Départements : TIC

Filière : Informatique et systèmes de communication

Orientation : Systèmes informatiques embarqués

Auteur : Tschantz Nathan

Supervisé par : Favrat Pierre & Mahboob Karimian

Date : 18 juillet 2026

1 Préambule

Ce travail de Bachelor (ci-après **TB**) est réalisé en fin de cursus d'études, en vue de l'obtention du titre de Bachelor of Science HES-SO en Ingénierie.

En tant que travail académique, son contenu, sans préjuger de sa valeur, n'engage ni la responsabilité de l'auteur, ni celles du jury du travail de Bachelor et de l'École.

Toute utilisation, même partielle, de ce TB doit être faite dans le respect du droit d'auteur.

HEIG-VD
Le Chef du Département

Yverdon-les-Bains, le 18 juillet 2026

2 Authentification

Je soussigné, Nathan Tschantz, atteste par la présente avoir réalisé seul ce travail et n'avoir utilisé aucune autre source que celles expressément mentionnées.

Nathan Tschantz

A handwritten signature in black ink, appearing to read 'N. Tschantz', written in a cursive style.

Yverdon-les-Bains, le 18 juillet 2026

3 Résumé - français

Contexte : Le pilotage de drones FPV en environnements complexes nécessite un retour vidéo en temps réel. Or, les fréquences standards (2.4 et 5.8 GHz) pénètrent mal les obstacles physiques (conditions NLOS).

Problématique : La norme Wi-Fi HaLow (IEEE 802.11ah, Sub-1 GHz) offre une excellente pénétration, mais ses mécanismes natifs de fiabilité (acquittements MAC, retransmissions) induisent une latence et une gigue incompatibles avec les exigences strictes du vol en immersion.

Méthode : Ce travail implémente une architecture de bout en bout optimisée pour le temps réel. Un programme en C développé sur la Raspberry Pi isole et fragmente activement les images d'un flux vidéo MJPEG natif en paquets autonomes, transmis via SPI à un microcontrôleur STM32. La liaison radio est configurée en UDP Unicast sur une modulation fixe et robuste (MCS 1, 8 MHz), tandis que les paramètres d'accès au canal de la Qualité de Service (QoS MAC) du modem sont forcés pour éliminer les temps d'attente inter-frames.

Résultat : Après avoir isolé un temps de vol radio pur de l'ordre de 12.5 ms, l'approche itérative a permis d'éliminer les lourds goulots d'étranglement applicatifs. La latence visuelle globale absolue (*Glass-to-Glass*) du système final s'établit entre **40 et 50 ms en moyenne**. Ces performances dépassent l'objectif d'acceptabilité de 80 ms et valident la viabilité du Wi-Fi HaLow pour le pilotage en milieu confiné.

4 Abstract - English

Context : FPV drone piloting in complex environments requires a real-time video feed. However, standard frequencies (2.4 and 5.8 GHz) struggle to penetrate physical obstacles (NLOS conditions).

Problem : The Wi-Fi HaLow standard (IEEE 802.11ah, Sub-1 GHz) offers excellent obstacle penetration, but its native reliability mechanisms (MAC acknowledgments, retransmissions) introduce latency and jitter incompatible with the strict requirements of immersive flight.

Method : This work implements an end-to-end architecture tailored for absolute real-time transmission. A custom C program running on the Raspberry Pi actively isolates and fragments native MJPEG video frames into autonomous packets, which are transmitted over SPI to a STM32 microcontroller. The radio link is configured using UDP Unicast on a robust, fixed modulation scheme (MCS 1, 8 MHz), while the modem's MAC Quality of Service (QoS) channel access parameters are forced to eliminate inter-frame waiting times.

Result : After isolating a pure radio transport time of approximately 12.5 ms, the iterative approach successfully eliminated heavy application-level bottlenecks. The absolute overall *Glass-to-Glass* visual latency of the final system ranges between **40 and 50 ms on average**. These performances surpass the initial 80 ms target and definitively validate the viability of Wi-Fi HaLow for drone piloting in confined environments.

Table des matières

1	Préambule	3
2	Authentification	5
3	Résumé - français	7
4	Abstract - English	7
5	Terminologie et concepts clés	13
	Terminologie et concepts clés	13
6	Introduction	16
6.1	Contexte	16
6.2	Problématique	16
6.3	Objectifs du travail de Bachelor	16
6.3.1	Définition du seuil d'acceptabilité et cas d'usage	17
6.4	Structure du document	17
7	Étude de faisabilité	18
7.1	But	18
7.2	Méthodologie d'investigation	18
7.3	Faisabilité côté Émetteur (FreeRTOS / STM32)	18
7.4	Faisabilité côté Récepteur (Linux / OpenWrt)	18
7.5	Conclusion de la faisabilité	19
8	Architecture globale du système	20
8.1	Le module d'acquisition et d'émission embarqué (TX)	20
8.2	Le pont réseau de réception au sol (routeur RX)	21
8.3	La station de décodage et d'affichage	21
8.4	Architecture finale retenue pour le système	22
8.5	Précision sur la structure du document	22
9	Configuration de l'environnement de développement embarqué	23
9.1	Intégration du SDK sous Windows	23
9.2	Adaptation de la chaîne de cross compilation	23
9.3	Configuration du débogage matériel (ST-LINK)	24
9.4	Provisionnement et calibration radio (Config Store)	24
9.4.1	Architecture de flashage	24
9.4.2	Résolution du conflit de signature et de version matérielle	24
10	Implémentation du réseau Wi-Fi HaLow	25
10.1	Contournement des contraintes réglementaires (ETSI vs FCC)	25
10.2	Stratégie de transport UDP : Du Broadcast à l'Unicast optimisé	25
10.2.1	L'approche initiale : UDP Broadcast et rétention DTIM	26

10.2.2	La solution finale : UDP Unicast et forçage de la QoS (WMM/EDCA)	26
10.3	Optimisation de la réactivité radio : Désactivation du PSM	27
10.4	Le routage et l'isolation IP (OpenWrt)	27
10.5	Validation de la bande passante radio : PHY vs Réelle	28
11	Capture et traitement du flux vidéo	29
11.1	La phase d'expérimentation : L'analyse des codecs inter-frames (H.264 et H.265)	29
11.1.1	Arbitrage initial entre H.264 et H.265	29
11.1.2	Limites physiques et latence plancher du H.264	29
11.2	La phase finale : Le changement de paradigme vers le MJPEG	30
11.2.1	Le prérequis indispensable : L'élargissement de la bande passante (BW)	30
11.2.2	Profilage du flux MJPEG et fragmentation réseau	30
11.3	Pipelines d'émission MJPEG final (TX)	32
11.3.1	Précisions sur les paramètres d'envoi	32
11.4	Pipeline de réception MJPEG final (RX)	33
11.4.1	Précisions sur les paramètres de réception	33
12	Interfaçage matériel et transfert SPI	34
12.1	Identification du bus SPI matériel	34
12.2	Développement de la Gateway SPI-MJPEG (C)	35
12.3	Contrôle de flux matériel (Handshake SPI)	35
12.4	Optimisation de l'ordonnancement OS et Overclocking	36
12.4.1	Overclocking et verrouillage des fréquences	36
12.4.2	Isolation du cœur CPU	36
13	Analyse des performances	37
13.1	Profilage théorique du transport matériel	37
13.1.1	Mesure au cycle d'horloge du traitement MCU (STM32)	37
13.1.2	Bilan de la latence de transport radio pur	37
13.2	Traque de la latence applicative : Le plafond de verre H.264	38
13.3	Performances et limites de l'architecture finale	38
13.3.1	Comparatif avec un système Broadcast alternatif (DVB-T2)	38
13.3.2	Validation matérielle RF et limitations de puissance	38
13.3.3	Évaluation empirique de la portée et de la pénétration (NLOS)	39
13.3.4	Qualité visuelle en conditions réelles	39
13.3.5	Bilan Glass-to-Glass final	42
14	Conclusion	45
15	Note sur l'utilisation de l'intelligence artificielle	46
16	Bibliographie	47
Index		49

Table des figures

1	Structure globale du système	17
2	Architecture globale du système de transmission vidéo avec les différentes options testées	20
3	Architecture retenue	22
4	Flux logique des données : (1) Lien réseau Sub-1 GHz, (2) Capture vidéo, (3) Plateformes de traitement, (4) Interfaçage SPI.	22
5	Configuration du serveur GDB ST-LINK dans STM32CubeIDE	24
6	(1) Lien réseau Sub-1 GHz	25
7	(2) Capture vidéo	32
8	(3) Plateformes de traitement.	33
9	(4) Interfaçage SPI	34
10	Schéma du système de test de latence matérielle	37
11	Flux vidéo FPV (MJPEG 320x240)	40
12	Référence smartphone	40
13	Flux FPV : Détail d'une plate-bande	40
14	Référence smartphone	40
15	Flux FPV : Obstacles au sol	41
16	Référence smartphone	41
17	Mesure de latence visuelle : 35 ms	42
18	Mesure de latence visuelle : 40 ms	43
19	Graphique de l'analyse statistique des 79 mesures réelles de latence Glass-to-Glass (Unicast MJPEG @ 60 FPS)	44

Liste des tableaux

1	Routage du bus SPI sur la carte d'évaluation EKH05	34
2	Portée utile et comportement de la liaison en mode NLOS	39

5 Terminologie et concepts clés

Afin de faciliter la lecture de ce document et la compréhension des choix architecturaux liés à la transmission vidéo basse latence, les acronymes et concepts techniques suivants seront couramment utilisés :

Compression Inter-frame : Méthode de compression vidéo temporelle (utilisée par le H.264 et le H.265) qui encode les différences entre plusieurs images consécutives. Bien qu'extrêmement économe en bande passante, elle exige la mise en mémoire tampon et l'analyse de dépendances temporelles de plusieurs trames au décodage, introduisant une latence logicielle structurelle incompatible avec le FPV.

Compression Intra-frame : Méthode de compression vidéo exclusivement spatiale (utilisée par le MJPEG) où chaque image est compressée de manière totalement autonome, à la manière d'une photographie JPEG isolée. Elle élimine instantanément le besoin de mise en mémoire tampon temporelle au décodage, permettant d'afficher l'image dès sa réception sur le réseau au prix d'un flux de données plus lourd.

Couche MAC : (*Media Access Control*) Sous-couche du modèle réseau OSI chargée de contrôler l'accès physique au support de transmission (l'air, dans notre cas), de réguler le partage du canal et de gérer les acquittements matériels.

Couche PHY : Niveau physique fondamental du modèle OSI. Elle gère la transformation des bits applicatifs en signaux électriques ou en ondes radiofréquences, dictant la modulation et la largeur de bande du canal.

DWT : (*Data Watchpoint and Trace*) Composant matériel intégré aux processeurs ARM Cortex-M permettant, entre autres, de compter les cycles d'horloge du cœur pour un profilage temporel avec une précision absolue de l'ordre de la nanoseconde.

EKH05 / EKH19 : Cartes d'évaluation (*Evaluation Kits*) développées par Morse Micro intégrant les puces Wi-Fi HaLow MM8108. L'EKH05 dispose d'entrées/sorties matérielles étendues (bus SPI, GPIO) pour l'IoT, tandis que l'EKH19 est conçue pour un interfaçage réseau standard (USB ou routeur OpenWrt).

FPV (*First Person View*) : « Vol en immersion ». Technique de pilotage où la caméra embarquée sur le drone transmet un flux vidéo en temps réel vers les lunettes ou l'écran du pilote, exigeant une latence extrêmement faible et constante.

FreeRTOS / RTOS : (*Real-Time Operating System*) Système d'exploitation temps réel open-source pour microcontrôleurs. Contrairement à un OS classique (comme Linux), un RTOS garantit l'exécution des tâches dans des délais stricts et déterministes, minimisant ainsi la gigue logicielle lors du routage des données.

Gigue (*Jitter*) : Variation temporelle de la latence. Dans un flux vidéo en temps réel, une forte gigue (des paquets arrivant en rafales irrégulières) provoque des saccades visuelles souvent plus perturbantes pour le pilotage qu'une latence fixe et stable.

GPIO : (*General Purpose Input/Output*) Broche physique d'un microcontrôleur pouvant être configurée par logiciel comme entrée ou sortie numérique (utilisée dans ce projet pour le signal de *Handshake*).

- GStreamer** : Framework multimédia open-source reposant sur une architecture de graphes (pipelines). Il permet de chaîner des éléments logiciels pour capturer, encoder, transmettre et décoder de la vidéo avec une latence minimale.
- Jetson Nano** : Nano-ordinateur développé par Nvidia, équipé d'un processeur ARM et d'un processeur graphique (GPU) intégré, utilisé lors des premières phases d'expérimentation du projet.
- Latence « Glass-to-Glass »** : Délai total mesuré entre l'instant où un événement physique se produit devant l'objectif de la caméra (le premier « verre ») et le moment où il est affiché sur l'écran du pilote (le second « verre »). C'est la métrique de performance absolue de ce projet.
- LDPC** : (*Low-Density Parity-Check*) Algorithme avancé de correction d'erreurs (*Forward Error Correction* - FEC). Il utilise des matrices de parité complexes permettant au récepteur de reconstruire des paquets radio corrompus par de mauvaises conditions RF, évitant ainsi d'avoir à les retransmettre.
- LwIP** : (*Lightweight IP*) Pile réseau TCP/IP open-source, hautement optimisée pour les microcontrôleurs disposant de ressources mémoire (RAM) limitées.
- MCS** : (*Modulation and Coding Scheme*) Indice définissant le débit physique de la transmission radio. Un MCS bas (ex : MCS 1) offre un débit réduit mais garantit une robustesse et une sensibilité maximales face au bruit et aux interférences.
- MM8108** : Puce SoC (*System on a Chip*) développée par Morse Micro. C'est le composant électronique au cœur de ce projet, chargé de la modulation et de l'émission des ondes Wi-Fi HaLow.
- NLOS** (*Non-Line-Of-Sight*) : Situation de communication radio où l'émetteur et le récepteur ne sont pas en ligne de vue directe (présence de bâtiments, de dalles en béton ou de relief).
- OpenHD** : Système d'exploitation open-source basé sur Linux, conçu par la communauté FPV pour transmettre de la vidéo numérique longue portée en détournant des puces Wi-Fi standards (2.4 et 5.8 GHz).
- OpenOCD** : (*Open On-Chip Debugger*) Logiciel libre agissant comme un serveur de débogage. Il permet de programmer et d'inspecter la mémoire d'un microcontrôleur depuis un PC via une interface matérielle.
- OpenWrt** : Distribution Linux open-source extrêmement légère, conçue pour remplacer les microprogrammes (*firmwares*) des routeurs réseau, offrant un contrôle bas niveau sur les interfaces de routage.
- Provisionnement** (*Provisioning*) : Processus technique consistant à injecter des données de configuration vitales, des fichiers de calibration matérielle (binaire de microcode BCF) et des profils réglementaires régionaux dans une zone mémoire non volatile dédiée (le *Config Store* du STM32), indispensable au démarrage de la puce radio MM8108.
- Raspberry Pi** : Gamme de nano-ordinateurs monocartes fonctionnant sous Linux, retenue comme plateforme finale d'acquisition vidéo grâce à l'efficacité de son outil natif `rpical-vid`.

SPI : (*Serial Peripheral Interface*) Bus de communication matériel synchrone fonctionnant en mode Maître/Esclave. Utilisé ici à une fréquence de 8 MHz pour le transfert à haute vitesse du flux vidéo compressé entre la Raspberry Pi et le microcontrôleur.

ST-LINK : Sonde de débogage matérielle développée par STMicroelectronics, permettant à un ordinateur de flasher du code et de contrôler un microcontrôleur STM32.

STM32 : Famille de microcontrôleurs 32 bits basés sur l'architecture ARM Cortex-M, développée par STMicroelectronics.

STM32CubeIDE : Environnement de développement intégré (IDE) officiel de STMicroelectronics, basé sur Eclipse, utilisé pour écrire, compiler et déboguer le code C du projet embarqué.

Tcpdump : Outil d'analyse réseau en ligne de commande permettant de capturer et d'inspecter les paquets de données (trames) transitant sur une interface réseau matérielle.

UDP Unicast & No-ACK : Stratégie de transmission réseau consistant à envoyer des paquets de données ciblés vers une unique adresse de destination, tout en modifiant les couches basses pour supprimer les mécanismes d'acquittement logiciel afin de privilégier un flux continu à latence minimale.

Wi-Fi HaLow (IEEE 802.11ah) : Amendement du standard Wi-Fi conçu pour opérer dans les bandes de fréquences Sub-1 GHz. Contrairement au Wi-Fi classique, il privilégie la portée et la capacité à traverser les obstacles physiques au détriment de la bande passante maximale.

6 Introduction

6.1 Contexte

Le pilotage de drones FPV (*First Person View*) repose sur un retour vidéo en temps réel. Pour garantir la sécurité du vol et la réactivité du pilote, la fiabilité de la liaison radio et la minimisation de la latence sont absolument critiques. Actuellement, la grande majorité des retours vidéo commerciaux reposent sur des liaisons fonctionnant dans les bandes de fréquences de 2.4 GHz ou 5.8 GHz. Bien que ces hautes fréquences offrent une large bande passante permettant le transfert de vidéos en haute définition, leur capacité de pénétration est en revanche médiocre face aux obstacles physiques.

En environnement urbain, forestier ou industriel (conditions dites *Non-Line-Of-Sight* ou NLOS), la portée de ces systèmes vidéo se trouve drastiquement réduite. À l'inverse, les protocoles opérant dans les bandes inférieures à 1 GHz (Sub-1 GHz, tels que TBS Crossfire ou ExpressLRS) sont devenus le standard industriel pour le lien de contrôle (la radiocommande) grâce à leur robustesse face aux obstacles, mais leur bande passante a toujours été techniquement insuffisante pour y faire transiter de la vidéo.

6.2 Problématique

Pour pallier ce manque et réunir le meilleur des deux mondes, la norme IEEE 802.11ah, également connue sous le nom de Wi-Fi HaLow, se présente comme une alternative prometteuse. Opérant dans la bande de fréquences Sub-1 GHz, elle offre une portée kilométrique et une excellente capacité à traverser les matériaux, tout en proposant des débits de l'ordre du Mégabit par seconde.

Cependant, les standards Wi-Fi classiques n'ont pas été conçus à l'origine pour transporter des flux vidéo continus et déterministes à très faible latence. Les mécanismes natifs d'adaptation de débit (*Rate Control*) et les retransmissions automatiques de paquets visent à garantir l'intégrité absolue des données au détriment du temps de livraison. Pour le vol FPV, l'attente d'une retransmission introduit une gigue (*jitter*) et une latence totalement incompatibles avec les exigences de pilotage.

6.3 Objectifs du travail de Bachelor

L'objectif principal de ce projet est d'étudier, de concevoir et de valider de bout en bout un prototype de transmission vidéo exploitant les avantages physiques de la norme IEEE 802.11ah (Wi-Fi HaLow).

Plutôt que de se limiter à la simple configuration logicielle de modules radio, l'enjeu s'est révélé être une véritable optimisation du système dans son ensemble. Cela implique le contournement des mécanismes d'acquiescement réseau (diffusion unidirectionnelle), le réglage strict des pipelines de compression vidéo matérielle, ainsi que la mise en place d'une synchronisation physique entre les différents processeurs et microcontrôleurs embarqués pour éviter l'accumulation de données dans les mémoires tampons (*buffers*).

6.3.1 Définition du seuil d'acceptabilité et cas d'usage

Au-delà de l'optimisation technique de la chaîne, il convient de définir un objectif de latence global pragmatique, aligné avec les contraintes physiques du vol en immersion. Sur une machine de type "freestyle" ou de course, les systèmes de transmission numériques traditionnels (opérant en 5.8 GHz) ciblent une latence de 20 à 30 ms (voire inférieure) pour autoriser des manœuvres très agressives.

Toutefois, l'objectif de ce travail n'est pas de concurrencer ces systèmes sur la vitesse pure, mais de privilégier la robustesse et la pénétration de signal en milieu complexe (conditions NLOS telles que l'exploration de bâtiments isolés, de forêts denses ou de tunnels). Dans ce contexte d'exploration, une absence de perte de liaison est préférable à une latence absolue minimale.

Le pilotage reste parfaitement viable et sécurisant jusqu'à un seuil de latence d'environ 80 ms. L'objectif de ce système, une fois toutes les optimisations matérielles et logicielles appliquées, est donc d'atteindre ce seuil des 80 ms, qui représente une limite raisonnable au vu de la chaîne d'acquisition matérielle grand public exploitée dans ce projet.

6.4 Structure du document

Ce rapport s'articule autour d'une progression logique, allant de la mise en place des fondations matérielles et logicielles jusqu'à l'optimisation itérative du système complet. Après une étude de faisabilité validant nos choix technologiques, nous détaillerons l'architecture globale retenue.



FIGURE 1 – Structure globale du système

Dans un premier temps, nous aborderons la configuration de l'environnement de développement et l'implémentation spécifique du lien réseau Wi-Fi HaLow, éléments qui constituent le socle de notre système.

Une fois ce pont réseau établi, nous décrirons les mécanismes logiciels de capture et de traitement de la vidéo, puis nous détaillerons l'interfaçage matériel (via le bus SPI) permettant d'acheminer ce flux du processeur vers le microcontrôleur d'émission.

Enfin, la dernière partie de ce rapport sera consacrée à l'approche itérative du projet : où seront présentées les différentes combinaisons de plateformes matérielles évaluées (Nvidia Jetson Nano, Raspberry Pi, Debix, PC) et où sera exposée l'analyse des mesures de latence ayant permis d'isoler les différents goulots d'étranglement de la chaîne.

7 Étude de faisabilité

7.1 But

Pour réaliser une transmission vidéo robuste face aux obstacles, notre stratégie repose sur le forçage d'un MCS bas et l'utilisation de la technique de correction d'erreurs LDPC. L'objectif de cette étude de faisabilité est de déterminer si ces paramètres, généralement gérés dynamiquement par le matériel, sont configurables manuellement sur les modules Wi-Fi HaLow Morse Micro MM8108 utilisés pour ce projet.

7.2 Méthodologie d'investigation

Dans un premier temps, l'analyse de la documentation constructeur et des interfaces utilisateur (*user-friendly*) d'OpenWrt a révélé que si certains paramètres basiques sont accessibles (comme le choix du canal ou de la largeur de bande), l'activation forcée du LDPC ou le verrouillage strict du MCS ne sont pas exposés par les utilitaires standards (tels que `iw`, `morse_cli` ou `hostapd`). L'investigation s'est donc orientée vers le reverse engineering et l'analyse du code source des pilotes fournis par Morse Micro, tant pour l'environnement FreeRTOS (Émetteur embarqué) que pour l'environnement Linux (Routeur relais) pour trouver le moyen de modifier ces paramètres.

7.3 Faisabilité côté Émetteur (FreeRTOS / STM32)

L'émetteur (TX), chargé de récupérer la vidéo et d'émettre les ondes radio depuis le drone, repose sur un microcontrôleur STM32 exploitant le système d'exploitation temps réel FreeRTOS. L'investigation du kit de développement logiciel (`mm-iot-sdk`) a permis d'identifier les leviers d'action suivants :

- **Contrôle du MCS** : L'analyse des bibliothèques a mis en évidence la fonction interne `mmwlan_ate_override_rate_control()` issue du fichier d'en-tête `mmwlan.h`. Cette fonction de test (ATE) permet de verrouiller dynamiquement la bande passante et le MCS directement dans le code source de l'application en C. C'est cette méthode applicative directe qui a été retenue et validée dans l'architecture finale pour imposer le MCS 1 à l'émetteur.
- **Activation du LDPC** : L'inspection des configurations internes, notamment celles liées au service `wpa_supplicant` (`config_ssid.h`), indique que la fonctionnalité LDPC est activée par défaut par le pilote lors de la négociation de la connexion.

7.4 Faisabilité côté Récepteur (Linux / OpenWrt)

Le relais radio, configuré en point d'accès (Access Point, AP) au sol pour réceptionner le flux, repose sur une architecture Linux (OpenWrt). L'analyse du code source du pilote noyau officiel de Morse Micro (`morse_driver`) a permis d'isoler les fichiers clés au sein du répertoire `/core/` :

- **Contrôle du MCS** : Les fichiers sources `core/rc.c` et `core/command.c` contiennent des structures permettant de désactiver l'algorithme d'adaptation de débit

(`enable_fixed_rate`) et d'imposer un MCS fixe (par exemple, MCS 2). Des tests de compilation croisée (*cross-compilation*) du noyau ont confirmé que ces modifications logicielles peuvent être compilées sous forme de module (`morse.ko`) et injectées à chaud sur le routeur cible.

- **La limite du forçage LDPC** : Initialement, une modification directe des entêtes de paquets MAC a été explorée. L'étude du fichier `skbq.c` laissait supposer la possibilité d'intercepter les trames pour manipuler le registre `morse_ratecode` (en forçant les bits B17 et B18 du champ SIG-1). Cependant, les expérimentations ont démontré qu'il est techniquement impossible d'appliquer et de maintenir ces paramètres matériels bas niveau tant que le routeur n'est pas formellement connecté à un client.

La preuve formelle d'une injection manuelle du LDPC n'a pas pu être clairement établie et a donc été écartée de l'implémentation finale. Néanmoins, la présence active des paramètres liés au LDPC et au MCS 10 dans le code source du pilote confirme la capacité de la puce à utiliser cette correction d'erreur en interne. Nous partons donc du postulat techniquement fondé que le contrôleur radio l'exploite de manière native.

7.5 Conclusion de la faisabilité

L'exploration en profondeur des pilotes Linux s'est révélée particulièrement intéressante pour comprendre les mécanismes de bas niveau et valider le comportement de la puce radio face aux contraintes de la norme 802.11ah. Néanmoins, les essais expérimentaux ont démontré qu'une modification du pilote noyau côté récepteur était superflue. Le standard Wi-Fi dictant que les paramètres de modulation physiques d'une trame sont imposés par l'émetteur, le forçage du MCS 1 appliqué directement via le code applicatif du STM32 (TX) suffit à verrouiller l'intégralité du lien radio descendant. L'objectif peut donc être atteint uniquement en configurant l'émetteur embarqué.

8 Architecture globale du système

Afin de répondre aux exigences de la transmission vidéo basse latence pour le pilotage FPV, l'architecture du système a été pensée de manière modulaire. La chaîne de transmission a été segmentée en trois entités fonctionnelles et matérielles distinctes :

1. **Le module d'acquisition et d'émission embarqué (TX)** : Destiné à être monté sur le drone, il est chargé de la capture vidéo brute, de sa compression, et de son émission radio sur la bande Sub-1 GHz.
2. **Le pont réseau de réception au sol (routeur RX)** : Il agit comme une passerelle réseau transparente. Son unique rôle est de réceptionner les ondes radio HaLow et de les convertir en trames Ethernet physiques, sans effectuer de traitement logiciel.
3. **La station de décodage et d'affichage** : Connectée au pont réseau via un câble Ethernet, cette entité est exclusivement dédiée au traitement logiciel du flux vidéo entrant pour l'afficher sur l'écran du pilote avec le délai le plus court possible.

Cette segmentation offre un double avantage : elle permet de décharger les microcontrôleurs Wi-Fi des lourdes tâches d'encodage/décodage vidéo, et elle a rendu possible le profilage de chaque sous-système séparément tout au long de la phase de développement. Voici l'ensemble des plateformes matérielles ayant été évaluées.

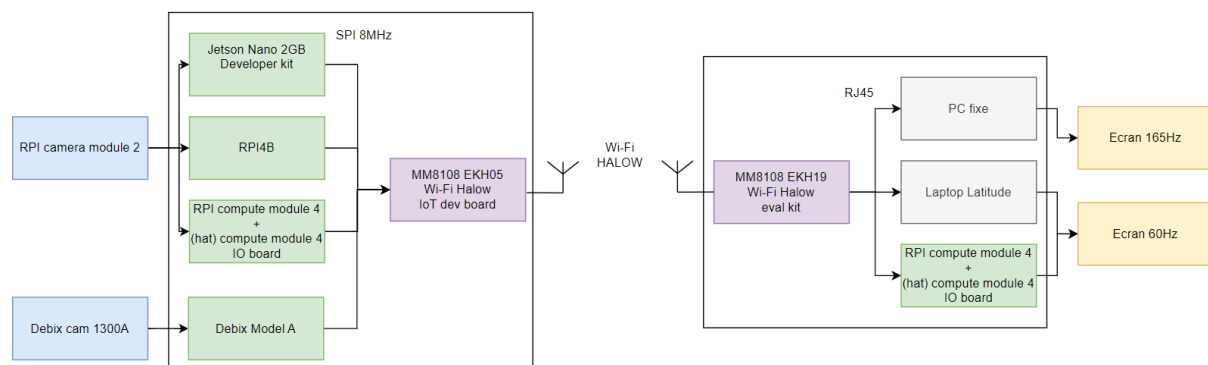


FIGURE 2 – Architecture globale du système de transmission vidéo avec les différentes options testées

8.1 Le module d'acquisition et d'émission embarqué (TX)

L'émetteur embarqué constitue le point de départ de la chaîne d'acquisition. Le choix du module de traitement vidéo et d'encodage a fait l'objet d'une sélection itérative parmi plusieurs ordinateurs monocartes :

- Une **Nvidia Jetson Nano 2GB** : Évaluée lors des premières phases d'expérimentation pour valider la chaîne de transmission avec un encodeur matériel dédié (NVENC). Cette plateforme s'est avérée logiquement moins optimale pour notre application spécifique que l'écosystème Raspberry Pi. Elle a donc été écartée suite à cette première phase de validation.

- L'écosystème **Raspberry Pi 4** (regroupant la *Compute Module 4* et la *4B* classique en une seule architecture de référence) : Sélectionné comme la solution matérielle finale pour l'émission. L'accès direct et hautement optimisé au processeur d'image (ISP) via l'outil natif `rpicam-vid` s'est révélé nettement plus efficace en termes de latence brute que toutes les autres solutions testées.
- Une **Debix Model A** : Évaluée brièvement pour ses capacités théoriques. Elle a rapidement été écartée car l'exploitation de son accélération matérielle nécessitait l'utilisation de longs pipelines GStreamer qui introduisaient plus de latence logicielle que l'implémentation directe et rationalisée de `rpicam-vid` sur la Raspberry Pi.

Indépendamment de l'ordinateur monocarte retenu pour la compression, les trames segmentées sont transmises via un bus SPI cadencé à 8 MHz vers un microcontrôleur **STM32U5**. Ce MCU agit comme un pont matériel déterministe. Il encapsule les données vidéo entrantes dans des paquets UDP à l'aide de la pile réseau LwIP, avant de les expédier au module radio **Morse Micro MM8108 (EKH05)** pour la transmission RF sur la bande Sub-1 GHz.

8.2 Le pont réseau de réception au sol (routeur RX)

La réception des ondes HaLow est assurée par un routeur **GL.iNet MT3000** équipé d'une carte d'évaluation **EKH19**. Ce routeur, fonctionnant sous le système d'exploitation OpenWrt, est configuré pour agir comme un simple pont logiciel (*Bridge*) transparent. Il réceptionne les trames 802.11ah sur son interface sans-fil et les injecte directement sur son interface Ethernet physique, limitant au maximum le temps de transit et le traitement par la pile TCP/IP interne du CPU.

8.3 La station de décodage et d'affichage

La station au sol est le dernier maillon de la chaîne, responsable du décodage et de l'affichage. Deux environnements ont été employés pour observer l'impact des couches d'abstraction d'affichage :

- **La station cible (Raspberry Pi 4 / CM4)** : Utilisée lors du prototypage pour évaluer des pipelines d'affichage KMS (*Kernel Mode Setting*) afin de s'affranchir de l'overhead de l'OS.
- **La station de décodage haute performance (PC fixe, GPU NVIDIA RTX 4070 Ti)** : Bien que l'architecture cible finale d'un tel système soit embarquée, cette plateforme de diagnostic a été retenue pour l'établissement de nos mesures de performance finales. Ce choix est principalement dicté par la nécessité d'utiliser un écran haute fréquence, en l'occurrence **165 Hz**. Cette caractéristique est indispensable pour éliminer le goulot d'étranglement temporel lié au taux de rafraîchissement d'un affichage standard (60 Hz), permettant de mesurer la latence glass-to-glass au plus proche de sa valeur réelle.

8.4 Architecture finale retenue pour le système

Suite aux phases d'évaluations itératives, l'architecture optimale et stabilisée retenue pour l'évaluation finale du système s'articule autour des composants matériels et logiciels suivants :

- **Côté Émetteur (TX) :** Une caméra Raspberry Pi V2 reliée à une Raspberry Pi 4B (CPU isolé sur le cœur 3, overclocké à 2.0 GHz). Le flux vidéo est capturé et compressé à la volée au format MJPEG à l'aide de l'outil `rpicas-vid` (320x240, 60 FPS, Qualité 8). Les trames sont découpées et poussées sur le bus SPI cadencé à 8 MHz par notre passerelle en C (`gateway_spi.c`) vers la carte de transmission STM32U5/MM8108 (EKH05).
- **Liaison Radio :** Modulation fixe forcée en MCS 1 (8 MHz de largeur de bande) utilisant le protocole UDP Unicast avec un marquage agressif de la Qualité de Service MAC (WMM/EDCA file "Voice") et la désactivation totale du mode d'économie d'énergie (PSM).
- **Côté Récepteur (RX) :** La carte de réception EKH19 reliée en bus interne au routeur OpenWrt GL.iNet MT3000, configurée en simple pont Ethernet transparent vers notre PC fixe de diagnostic (décodage GStreamer logiciel sans synchronisation et affichage direct sur l'écran haute vitesse 165 Hz).

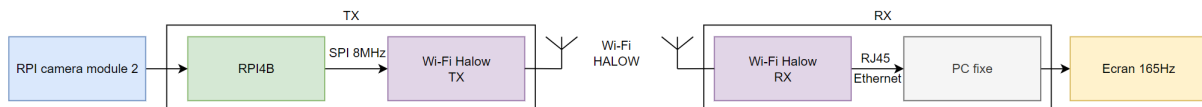


FIGURE 3 – Architecture retenue

8.5 Précision sur la structure du document

Comme vu précédemment voici l'ordre dans le quel les différents éléments du système seront développés.

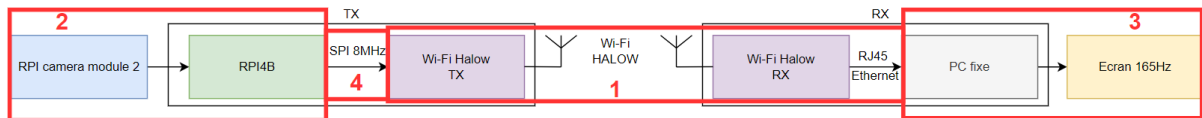


FIGURE 4 – Flux logique des données : (1) Lien réseau Sub-1 GHz, (2) Capture vidéo, (3) Plateformes de traitement, (4) Interfaçage SPI.

9 Configuration de l'environnement de développement embarqué

Le développement du firmware pour le microcontrôleur gérant la transmission (STM32) repose sur le kit de développement logiciel officiel de Morse Micro (`mm-iot-sdk`). Ce SDK étant nativement conçu pour un environnement Linux avec Platformio et des chaînes de cross compilation préinstallées, son intégration sous Windows a nécessité plusieurs adaptations.

9.1 Intégration du SDK sous Windows

Le code source a été récupéré via le dépôt GitHub officiel. L'architecture du projet s'appuyant sur plusieurs dépendances externes vitales (telles que le système d'exploitation temps réel FreeRTOS, la pile réseau LwIP et mbedTLS), une initialisation complète des sous-modules a été requise (`git submodules`).

L'environnement de développement choisi est **STM32CubeIDE**. Le SDK ne respectant pas la structure standard de cet IDE, le projet a été importé en tant que "Makefile Project with Existing Code", en ciblant spécifiquement le répertoire de la carte d'évaluation de l'émetteur (`mm-mm6108-ekh05` avant sa mise à jour, `mm-mm8108-ekh05` après).

9.2 Adaptation de la chaîne de cross compilation

Sous Windows, les outils de compilation en ligne de commande tels que `make` ne sont pas natifs. Pour pallier ce problème, les variables d'environnement du projet dans STM32CubeIDE ont été modifiées pour inclure les chemins absolus vers les utilitaires internes de l'IDE (`make.exe` et `arm-none-eabi-gcc.exe`).

De plus, le SDK force la recherche de la chaîne de compilation GCC ARM dans les répertoires typiques de Linux (tels que `/opt/`). Il a donc été nécessaire de modifier le fichier de configuration principal `mkcore-arm-cortex-m33f.mk` pour empêcher cette recherche et permettre au système d'utiliser le `PATH` de Windows :

```
# --- MODIFICATION POUR WINDOWS / STM32CubeIDE ---
# On ignore la recherche des dossiers Linux /opt/...
# et on laisse le PATH systeme de STM32CubeIDE trouver le
#   compilateur.
TOOLCHAIN_DIR :=
TOOLCHAIN_BASE := arm-none-eabi-

CC := "$(TOOLCHAIN_BASE)gcc"
CXX := "$(TOOLCHAIN_BASE)g++"
AS := $(CC) -x assembler-with-cpp
OBJCOPY := "$(TOOLCHAIN_BASE)objcopy"
```

9.3 Configuration du débogage matériel (ST-LINK)

Pour permettre l'exécution pas à pas et la pose de breakpoint, le projet a été converti en "Projet ARM" dans l'IDE. La sonde de débogage utilisée est le **ST-LINK** intégré à la carte EKH05, communiquant via le protocole SWD (*Serial Wire Debug*).

FIGURE 5 – Configuration du serveur GDB ST-LINK dans STM32CubeIDE

Il est important de noter une limitation physique de cette architecture : la sonde USB (ST-LINK) ne peut maintenir qu'un seul tunnel de communication actif à la fois. L'utilisation simultanée de la console série (ex : PuTTY, Termit) et du débogueur GDB entraîne des conflits matériels.

9.4 Provisionnement et calibration radio (Config Store)

Une étape indispensable, distincte de la compilation du code C, est le **provisionnement** de la puce Wi-Fi. La puce Morse Micro ne possède pas de mémoire non volatile interne pour ses paramètres vitaux ; elle ne peut pas démarrer sans son micro-code (BCF) et ses paramètres régionaux (*Country Code*), stockés dans une zone mémoire spécifique du STM32 appelée le *Config Store*.

9.4.1 Architecture de flashage

Le processus de flashage repose sur une architecture client-serveur :

1. **Le Serveur (OpenOCD)** : Il établit la connexion matérielle entre le port USB et le processeur ARM via la sonde ST-LINK.
2. **Le Client (Script Python)** : Il lit le fichier de configuration utilisateur (`config.hjson`), génère le binaire correspondant, et commande au serveur de l'écrire dans la mémoire Flash du STM32 (à l'adresse `0x08004000`).

9.4.2 Résolution du conflit de signature et de version matérielle

Lors des premiers déploiements, une erreur bloquait la communication SPI entre le MCU et la puce Wi-Fi (`Transport init failed - Chip ID 0x0000`). Le code embarqué plan-tait dès l'initialisation du HAL (Hardware Abstraction Layer).

L'analyse de ce blocage a révélé un désaccord matériel : le code C et le fichier de configuration cachaient une dépendance stricte à l'ancienne plateforme (MM6108). Notre matériel étant la version MM8108, le binaire de calibration injecté était incorrect. Par une coïncidence extrêmement heureuse, Morse Micro a mis à jour son SDK public la semaine suivante, ajoutant officiellement le support de la cible `mm-mm8108-ekh05`.

Cette mise à jour a permis de rediriger le script vers le fichier de calibration exact de la puce (`bcbf_mf08651_us.mbin`). Combiné à l'ajustement du fichier `config.hjson` pour cibler la région "US", la puce radio a pu démarrer avec succès.

10 Implémentation du réseau Wi-Fi HaLow

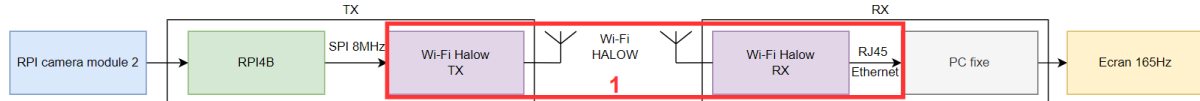


FIGURE 6 – (1) Lien réseau Sub-1 GHz

La norme IEEE 802.11 (Wi-Fi) intègre intrinsèquement des mécanismes de fiabilité, tels que les accusés de réception (ACK) matériels au niveau de la couche MAC et les retransmissions automatiques. Si ces fonctionnalités garantissent l'intégrité des données pour de l'IoT standard, elles s'opposent aux exigences du flux vidéo FPV où la perte d'une trame est largement préférable à l'attente d'une retransmission tardive. L'ensemble de la pile réseau a donc été reconfiguré pour prioriser la réactivité au détriment de la fiabilité de transport.

10.1 Contournement des contraintes réglementaires (ETSI vs FCC)

Avant d'entamer la configuration fine du flux vidéo, l'établissement de la liaison radio HaLow en Europe (norme ETSI, bande des 868 MHz) s'est heurté à un obstacle majeur : une latence de base colossale d'environ 670 ms, mesurée lors de simples paquets de test (Ping).

L'analyse de ce délai a mis en évidence l'impact restrictif des réglementations européennes régissant les bandes de fréquences Sub-1 GHz. Pour permettre la coexistence des équipements, l'ETSI impose le mécanisme *Listen Before Talk* (LBT) ainsi que des restrictions strictes de *Duty Cycle* (temps d'émission maximal autorisé cumulé). Le module radio Morse Micro bridait donc son propre débit et retenait délibérément les paquets pour respecter ces contraintes légales.

Afin de mesurer les performances physiques réelles du matériel sans ces limitations logicielles régionales, l'ensemble du système a été basculé sur le domaine réglementaire américain (FCC, bande des 915 MHz). L'utilisation du Canal 28 (916 MHz) permet de désactiver les mécanismes LBT et de s'affranchir des limites de *Duty Cycle*. Ce choix expérimental indispensable a permis de faire chuter instantanément la latence de base du ping à sa valeur matérielle brute, située entre **3 et 5 ms**, rendant le projet possible.

10.2 Stratégie de transport UDP : Du Broadcast à l'Unicast optimisé

La couche transport exploite le protocole UDP pour s'affranchir de la gestion des connexions et des retransmissions inhérentes à TCP. L'élimination des délais au niveau de la couche liaison (MAC 802.11ah) a quant à elle nécessité deux approches successives.

10.2.1 L'approche initiale : UDP Broadcast et rétention DTIM

Pour écarter la contrainte d'attente des acquittements matériels (MAC ACKs), la première stratégie consistait à émettre le flux vidéo sur l'adresse de diffusion locale (192.168.12.255). LwIP traduit cette adresse par l'adresse MAC de destination globale FF:FF:FF:FF:FF:FF. Selon la norme 802.11, les trames diffusées en *Broadcast* ne déclenchent aucun acquittement de la part des récepteurs.

Cependant, les tests d'émission ont révélé un goulot d'étranglement majeur induit par le routeur OpenWrt de réception. Selon les spécifications de la norme IEEE 802.11, les points d'accès ont l'obligation réglementaire de mettre en mémoire tampon les trames de diffusion (*Broadcast/Multicast*) et de ne les libérer qu'immédiatement après la transmission d'une balise de signalisation spécifique appelée DTIM (*Delivery Traffic Indication Message*). Ce mécanisme est conçu à l'origine pour permettre aux stations IoT de rester en veille profonde et de ne se réveiller que périodiquement pour écouter le trafic global.

Avec un intervalle de balise par défaut de 100 ms et un paramètre DTIM configuré à 2 sur OpenWrt, le point d'accès ne libérait les données accumulées que toutes les 200 ms. L'analyse des captures a mis en évidence une rétention logicielle asynchrone induisant une latence et une gigue colossales d'environ **40 ms**. Le passage en Unicast a permis de contourner immédiatement ce tampon pour envoyer chaque paquet en flux tendu.

10.2.2 La solution finale : UDP Unicast et forçage de la QoS (WMM/EDCA)

Pour contourner la rétention liée au DTIM, la liaison a été basculée en **UDP Unicast**, ciblant directement l'adresse IP unique de la station de réception (192.168.12.10). En Unicast, le point d'accès OpenWrt transmet immédiatement chaque paquet reçu sur l'interface radio.

Le retour à l'Unicast réintroduisant obligatoirement le mécanisme d'acquittement matériel (MAC ACK) sous peine de voir la puce radio couper la transmission, il a fallu configurer le modem de manière agressive au niveau de ses files d'attente de priorité d'accès au canal (EDCA/WMM).

Dans un premier temps, les paquets UDP vidéo sont marqués au niveau IP avec un drapeau TOS (*Type of Service*) prioritaire (`pcb->tos = 0xC0`, correspondant à la catégorie réseau *Voice*). Dans un second temps, la structure de contrôle du SDK Morse Micro a été configurée et forcée dans le firmware du STM32 :

```
// Configuration QoS ultra-agressive des files d'attente EDCA
struct mmwlan_qos_queue_params fpv_qos = {
    .aci = 3,                // Access Category Index : 3 correspond a la
                             // file d'attente "Voice"
    .aifs = 2,               // Arbitration Inter-Frame Spacing : Temps d'
                             // attente minimal legal
    .cw_min = 1,             // Contention Window Min : Taille minimale de
                             // la fenetre d'attente
    .cw_max = 1,             // Contention Window Max : Evite l'
                             // augmentation du delai apres collision
    .txop_max_us = 0         // Transmission Opportunity : Temps d'
                             // occupation maximal (0 = un seul paquet)
```

```
};  
mmwlan_set_default_qos_queue_params(&fpv_qos, 1);
```

L'explication physique de ce forçage est la suivante :

- **aci = 3 (Voice)** : Oriente physiquement nos paquets vidéo vers la file d'attente matérielle la plus prioritaire du modem radio.
- **aifs = 2** : Réduit le temps d'attente obligatoire entre deux tentatives d'émission à sa limite légale absolue. Le transmetteur écoute à peine le canal avant de parler.
- **cw_min = 1 et cw_max = 1** : En Wi-Fi standard, si le canal est occupé ou qu'une collision survient, l'émetteur applique un recul aléatoire exponentiel pour retenter sa chance plus tard. En forçant ces fenêtres à leur valeur minimale (1), le STM32 retente l'émission immédiatement sans délai d'attente, écrasant virtuellement les autres trafics environnants pour imposer l'envoi de la vidéo.

L'application de ce "hack" de QoS a permis de digérer l'overhead des acquittements MAC Unicast tout en récupérant les **40 ms** perdues dans les buffers DTIM du routeur, stabilisant ainsi la liaison radio. Cela a également eu pour effet

10.3 Optimisation de la réactivité radio : Désactivation du PSM

Par défaut, le SDK Morse Micro active les profils d'économie d'énergie (*Power Saving Mode* - PSM) destinés aux capteurs autonomes de l'Internet des objets (IoT). Ce mode place temporairement le récepteur radio en veille profonde et nécessite une phase de réveil lors de chaque transaction réseau, ce qui générerait une latence systématique d'environ **22 ms** par paquet.

Le mode veille a donc été désactivé dans l'initialisation de l'application :

```
mmwlan_set_power_save_mode(MMWLAN_PS_DISABLED);
```

Ce forçage maintient la radio active à 100% de son temps, ramenant la gigue et le délai de transmission à leur minimum.

10.4 Le routage et l'isolation IP (OpenWrt)

Afin de simplifier le transit réseau, le réseau FPV est strictement isolé sur le sous-réseau 192.168.12.x.

Pour maximiser l'efficacité du routeur de réception au sol (GL.iNet MT3000 fonctionnant sous OpenWrt), la redirection des données en espace utilisateur (*User-Space*) via des utilitaires comme **netcat** ou des interceptions **tcpdump** a été proscrite en raison de la surcharge CPU qu'elle générerait. Un pont réseau (*Bridge*) a été configuré directement au niveau du noyau Linux du routeur :

```
brctl addif br-ahwlan wlan0
```

Ce pont matériel permet aux fragments vidéo UDP d'être répliqués de l'interface radio vers le port Ethernet RJ45 de manière transparente, sans aucune analyse ou rétention par les couches applicatives du routeur, économisant ainsi 10 à 20 ms de transit logiciel.

10.5 Validation de la bande passante radio : PHY vs Réelle

La désactivation des algorithmes d'adaptation automatique de débit (*Rate Control*) du modem radio a été configurée via la fonction ATE (*Automated Test Equipment*) du SDK :

```
mmwlan_ate_override_rate_control(MMWLAN_MCS_1 , MMWLAN_BW_8MHZ ,  
MMWLAN_GI_NONE);
```

En forçant l'utilisation du protocole sur le **MCS 1** avec une largeur de bande de **8 MHz**, la couche physique (PHY) affiche un débit de synchronisation théorique de ****6.4 Mbps**** (en Short Guard Interval). Il est cependant crucial de distinguer ce débit brut de la bande passante utile réellement disponible pour les données utilisateur (*data payload*).

En raison des en-têtes de protocoles empilés (PHY, MAC 802.11ah, IP, UDP), des temps d'attente inter-trames obligatoires (IFS) et surtout de la réintroduction des acquittements MAC en mode Unicast, le débit utile réel disponible au niveau applicatif s'effondre à **moins de 50 %** de la bande passante PHY théorique (soit environ 2.5 à 3 Mbps dans des conditions parfaites de laboratoire). En situation de vol réel avec un rapport signal-sur-bruit (SNR) dégradé, cette limite utile s'approche des **1.5 Mbps**. Ce constat justifie scientifiquement le dimensionnement de notre flux MJPEG, volontairement contraint et profilé sous ce seuil critique pour éviter toute saturation du canal.

11 Capture et traitement du flux vidéo

Une fois le pont réseau Sub-1 GHz configuré sur ses couches basses, le défi consiste à formater la source vidéo pour qu'elle puisse traverser ce canal sans générer de goulots d'étranglement applicatifs. Cette section détaille l'arbitrage entre les codecs, le profilage des flux et l'optimisation des pipelines.

11.1 La phase d'expérimentation : L'analyse des codecs inter-frames (H.264 et H.265)

L'étude initiale du projet s'est orientée vers les standards de compression vidéo modernes, caractérisés par un encodage temporel (*inter-frame*).

11.1.1 Arbitrage initial entre H.264 et H.265

La première architecture évaluée ciblait la norme **H.265 (HEVC)**. Ce codec offre théoriquement une efficacité de compression supérieure de 50 % par rapport au H.264, permettant de conserver une image exploitable à un débit extrêmement bas (400 kbps). Cependant, sa complexité algorithmique s'est heurtée à la réalité matérielle : l'absence d'un pilote de décodage matériel H.265 stable et performant sous Linux (`v4l2h265dec`) a provoqué une surcharge CPU massive sur nos plateformes, élevant la latence à plus de 300 ms.

Le système a donc été basculé vers la norme **H.264 (AVC)**. Ce choix a permis d'exploiter les puces de décodage matériel universelles du PC fixe de diagnostic et des Raspberry Pi, ramenant le temps de décodage en dessous des 50 ms.

11.1.2 Limites physiques et latence plancher du H.264

Malgré l'optimisation drastique des pipelines GStreamer à l'émission et à la réception (désactivation des B-Frames, profils *baseline*, forçage du mode *flush* pour vider les tampons), le système H.264 a buté sur un plafond de verre.

Les meilleures mesures de latence globale (*Glass-to-Glass*) obtenues avec ce codec se sont établies à :

- **Entre 100 ms et 120 ms** sur le PC fixe de diagnostic (bénéficiant du décodage GPU optimal et d'un affichage 165 Hz).
- **Entre 130 ms et 140 ms** sur la station de réception cible Raspberry Pi 4.

Cette latence a révélé un problème structurel : par définition, le H.264 est un codec **inter-frame**. Même configuré pour de l'ultra-basse latence, le décodeur et le parseur (`h264parse`) exigent de mettre en mémoire tampon et d'analyser les dépendances temporelles de plusieurs frames consécutives pour reconstruire l'image. Ce délai d'accumulation logiciel, couplé à l'overhead des files d'attente du système d'exploitation, n'est pas optimal pour du FPV.

11.2 La phase finale : Le changement de paradigme vers le MJPEG

Pour éliminer tout délai d'accumulation temporelle, la compression inter-frame a été abandonnée au profit du format **MJPEG** (*Motion JPEG*).

11.2.1 Le prérequis indispensable : L'élargissement de la bande passante (BW)

Le MJPEG est un codec exclusivement **intra-frame** : chaque image est compressée de manière autonome, comme une photographie JPEG isolée. Ce choix élimine instantanément le besoin de mise en mémoire tampon temporelle au décodage : une image reçue par le réseau est immédiatement affichable.

Cependant, cette absence d'analyse temporelle se traduit par un flux de données nettement plus lourd que le H.264. Lors de nos premiers essais en H.264, la bande passante radio était volontairement bridée sur une largeur de bande de **2 MHz** (MCS 2) pour maximiser la portée, ce qui limitait le débit utile à environ 400 Kbps. Faire passer un flux MJPEG dans un tel canal était physiquement impossible et causait une saturation immédiate.

La transition vers le MJPEG a donc été rendue possible uniquement par la reconfiguration matérielle de la liaison radio HaLow, basculée sur une largeur de bande de **8 MHz** (MCS 1). Ce changement de canal a permis de lever le goulot d'étranglement de la couche physique, offrant le débit nécessaire pour soutenir les exigences du format JPEG sans perte de paquets.

11.2.2 Profilage du flux MJPEG et fragmentation réseau

Afin de quantifier précisément les besoins en bande passante du flux MJPEG final, une mesure de métrologie systématique a été menée. Un enregistrement continu d'une minute (60 secondes) a été capturé en conditions réelles en agitant la caméra pour simuler le comportement dynamique d'un vol :

```
rplicam-vid -t 60000 -n -o fpv_test_1min.mjpeg --width 320 --height 240 --framerate 60 --codec mjpeg --denoise off --exposure sport --metering centre --awb daylight --quality 8 --flush
```

L'analyse du fichier binaire obtenu a permis d'établir le profilage mathématique suivant :

1. **Volume de données global** : Le fichier généré pèse exactement 11 534 336 octets (≈ 11.53 Mo).
2. **Volume d'images produit** : À une fréquence d'échantillonnage de 60 Hz sur une durée de 60 secondes, la caméra a produit exactement 3600 images.
3. **Poids moyen d'une frame (Frame Size)** :

$$\text{Taille moyenne} = \frac{11\,534\,336 \text{ octets}}{3600 \text{ images}} \approx \mathbf{3204} \text{ octets par image}$$

4. **Bande passante requise (Throughput)** : Le débit réel consommé par le flux vidéo se calcule en divisant le poids total par le temps d'acquisition :

$$\text{Bande passante} = \frac{11.53 \text{ Mo} \times 8 \text{ bits}}{60 \text{ s}} \approx \mathbf{1.53} \text{ Mbps}$$

Cette résolution (320x240) et ce framerate (60 fps) ont été trouvés de manière empirique ; c'est le « sweetspot » auquel la vidéo reste fluide avec une qualité acceptable pour l'utilisation. Au-delà de ces paramètres, des pertes sont observées ; on peut donc en déduire que 1.53 Mbps est la limite de charge utile du système.

Ce profilage démontre que l'image MJPEG moyenne (3204 octets) dépasse la taille maximale de la charge utile de nos paquets (MTU applicative fixée à 1400 octets, donc légèrement inférieur à la taille maximale d'une trame Ethernet standard qui est de 1500 octets). Une image est donc segmentée en moyenne en **3 paquets UDP** ($3204/1400 \approx 2.28 \rightarrow 3$ paquets).

11.3 Pipelines d'émission MJPEG final (TX)

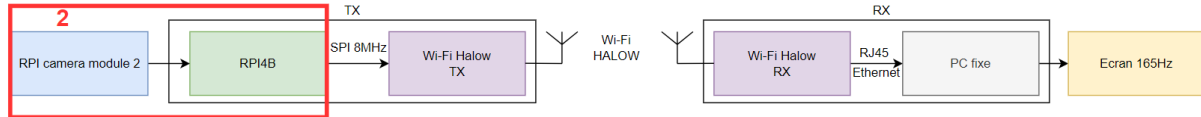


FIGURE 7 – (2) Capture vidéo

Pour implémenter cette stratégie MJPEG ultra-basse latence, le pipeline d'émission s'appuie sur l'outil natif `rpicam-vid` sur la Raspberry Pi (TX), configuré de manière extrêmement agressive pour garantir un temps de calcul déterministe de l'ISP :

```
# Commande d'acquisition et de compression MJPEG finale (Raspberry
  Pi TX)
rpicam-vid -t 0 -n -o - --width 320 --height 240 --framerate 60 --
  codec mjpeg \
--denoise off --exposure sport --metering centre --awb daylight --
  quality 8 --flush
```

Note : les paramètres optimaux pour le H.264 peuvent être retrouvés dans le rapport intermédiaire.

11.3.1 Précisions sur les paramètres d'envoi

Les arguments passés à la ligne de commande visent à supprimer tout traitement d'image adaptatif ou asynchrone :

- **-flush** : Force l'expulsion immédiate des données JPEG compressées de l'espace noyau vers la sortie standard (*stdout*), court-circuitant les tampons de cache en écriture du système d'exploitation de la Raspberry Pi.
- **-width 320 -height 240 (Downscaling matériel)** : L'ISP du capteur IMX219 effectue un sous-échantillonnage matériel direct sur la matrice de pixels. Cela réduit drastiquement le poids des données à traiter sans solliciter le processeur principal de la carte.
- **-denoise off** : Désactive l'algorithme de réduction du bruit numérique, éliminant ainsi une étape de calcul asynchrone sur l'ISP.
- **-exposure sport et -awb daylight** : Gèle l'exposition automatique sur des temps d'obturation très courts (*sport*) et fixe la balance des blancs. Cela évite les variations du temps de calcul de l'ISP lors des changements brusques de luminosité en vol.
- **-quality 8** : Règle la table de quantification JPEG sur une compression forte. Ce paramètre clé maintient le flux utile moyen autour de 1.5 Mbps, lui permettant de traverser le canal radio de 8 MHz sans saturation.

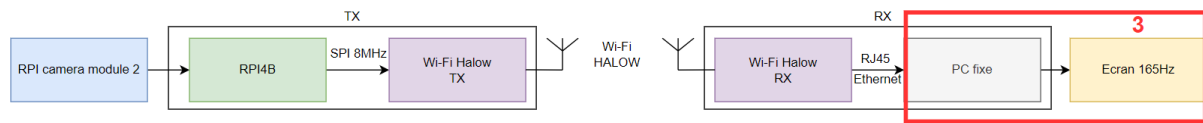


FIGURE 8 – (3) Plateformes de traitement.

11.4 Pipeline de réception MJPEG final (RX)

À la réception, le flux UDP Unicast arrivant sur le port de la station est décodé et affiché via le pipeline GStreamer épuré de tout mécanisme de synchronisation ou de lissage temporel :

```
# Commande de reception et d'affichage MJPEG finale (PC fixe de
  diagnostic)
gst-launch-1.0 -v udpsrc port=1337 do-timestamp=true ! \
jpegparse ! jpegdec ! queue max-size-buffers=1 leaky=downstream ! \
videoconvert ! autovideosink sync=false
```

11.4.1 Précisions sur les paramètres de réception

- **jpegparse (Réassemblage des fragments)** : Cet élément est indispensable. Le MJPEG générant plusieurs fragments UDP par image, le parseur réassemble les paquets de 1400 octets jusqu'à la détection du marqueur de fin d'image JPEG (0xFF 0xD9), transmettant l'image complète et propre au décodeur jpegdec pour éviter tout crash de l'affichage.
- **queue max-size-buffers=1 leaky=downstream** : Limite le stockage en mémoire à une unique image. Si le décodeur subit un retard de traitement, les anciennes images en attente sont immédiatement jetées en aval, forçant l'affichage à recoller instantanément au temps réel sans accumuler de décalage temporel (*buffer bloat*).
- **sync=false** : Désactive la synchronisation du moteur de rendu sur l'horloge globale de GStreamer, affichant l'image immédiatement après sa reconstruction en ignorant les métadonnées temporelles.

12 Interfaçage matériel et transfert SPI

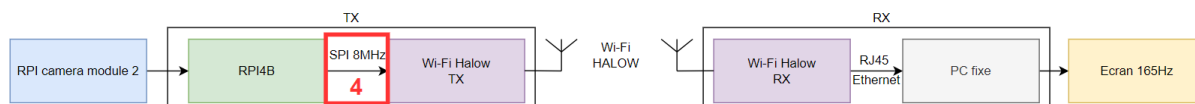


FIGURE 9 – (4) Interfaçage SPI

Le transfert du flux vidéo compressé entre le processeur d'acquisition (Raspberry Pi TX) et le modem radio (STM32) s'effectue via le bus matériel SPI. Cette étape est critique car elle conditionne le rythme d'envoi des données au MCU avant l'émission RF.

12.1 Identification du bus SPI matériel

Une attention particulière a dû être portée à la configuration physique de la liaison. Le SDK Morse Micro ne supportant nativement aucun périphérique standard en dehors de la puce Wi-Fi, le routage exact du bus SPI du microcontrôleur sur le connecteur d'extension n'était pas documenté dans le guide utilisateur de base.

Pour résoudre cette inconnue, une analyse du schéma de conception de la carte **EKH05** [7] a été menée afin d'identifier théoriquement les broches physiques et les pistes reliées aux périphériques internes du processeur. Afin de valider de manière absolue ce schéma avant d'y connecter l'ordinateur d'acquisition, une vérification a été effectuée : une tension de 3,3 V a été appliquée sur les lignes d'entrée repérées, permettant de confirmer au multimètre la correspondance exacte du routage (1).

Broche EKH05	Port STM32	Signal SPI	Description fonctionnelle
D10	PE12	NSS (nCS)	<i>Not Chip Select</i> (Sélection de l'esclave)
D11	PE15	MOSI	<i>Master Out Slave In</i> (Données entrantes)
D12	PE14	MISO	<i>Master In Slave Out</i> (Données sortantes)
D13	PE13	SCK	<i>Serial Clock</i> (Horloge)

TABLE 1 – Routage du bus SPI sur la carte d'évaluation EKH05

12.2 Développement de la Gateway SPI-MJPEG (C)

Le changement de paradigme vers le MJPEG a nécessité la réécriture complète du programme d'interfaçage en C (situé entre l'outil de capture et le bus SPI). En H.264, le flux pouvait être "poussé" aveuglément. En MJPEG, envoyer des morceaux bruts aléatoires à travers le réseau causait des plantages du décodeur GStreamer à la réception, qui ne tolère aucune corruption de structure.

Le programme d'émission (voir code complet en **Annexe A**) agit désormais comme un véritable parseur actif :

1. **Isolation des images** : Il lit le flux continu depuis l'OS en mode non-bloquant et scanne les octets à la volée pour identifier les marqueurs standards du protocole JPEG : 0xFF 0xD8 (Début d'image - SOI) et 0xFF 0xD9 (Fin d'image - EOI).
2. **Slicing à 1400 octets** : Une fois une image complète isolée en RAM, il la fragmente strictement en paquets de 1400 octets. Une capture réseau (`tcpdump`) a permis de valider empiriquement que l'application respectait bien cette taille maximale.
3. **Padding de sécurité** : Si le dernier morceau de l'image fait moins de 1400 octets, le programme complète l'espace vide du tampon SPI avec des zéros (0x00). Le vrai marqueur de fin d'image (0xFF 0xD9) généré par la caméra étant déjà inclus et préservé dans les données utiles, le parseur GStreamer de réception (`jpegparse`) reconstruit la trame et ignore naturellement ce padding neutre sans provoquer d'erreur.

12.3 Contrôle de flux matériel (Handshake SPI)

Le transfert s'effectue sur le bus SPI à une fréquence d'horloge poussée à **8 MHz**. Mathématiquement, l'envoi d'un paquet UDP contenant une charge utile de 1400 octets (soit 11 200 bits) à cette fréquence requiert un temps de transmission physique incompressible. Ce temps théorique se calcule ainsi :

$$T_{SPI} = \frac{1400 \times 8 \text{ bits}}{8\,000\,000 \text{ bits/s}} = 0.0014 \text{ s} = \mathbf{1.40 \text{ ms}}$$

L'architecture UDP étant configurée sans acquittement logiciel, si l'ordinateur enchaîne l'envoi de ces paquets de 1.40 ms trop rapidement, la mémoire RAM du microcontrôleur STM32 déborde (*Buffer Overrun*). Des morceaux d'images sont écrasés avant même d'atteindre le module Wi-Fi, provoquant de sévères artefacts visuels (effets de "bavure" ou de déchirement d'image).

Pour pallier ce problème, un mécanisme de *Handshake* matériel a été implémenté via une broche de signalisation dédiée (GPIO 24 sur la Raspberry Pi vers PD15 sur le STM32) :

```
// Attente active que le STM32 soit PRET (Broche a 1)
while ((* (gpio + 13) & (1 << GPIO_PIN)) == 0) {
    timeout_counter++;
    if (timeout_counter > 2000000) { return 0; }
}
```

Le processus est le suivant : le STM32 abaisse ce signal (niveau BAS) dès qu'il reçoit un paquet de 1400 octets via l'interruption SPI. Il ne le relève (niveau HAUT) que lorsque la couche réseau LwIP a fini de transférer le paquet au module Wi-Fi HaLow. Cette synchronisation matérielle garantit un flux vidéo propre, dictant la cadence de l'émetteur sans aucune perte interne au système.

12.4 Optimisation de l'ordonnancement OS et Overclocking

Afin de garantir un comportement strictement déterministe et d'éliminer la gigue (*jitter*) introduite par l'ordonnanceur multitâche du noyau Linux, deux optimisations système agressives ont été appliquées sur la Raspberry Pi (TX).

12.4.1 Overclocking et verrouillage des fréquences

Par défaut, le noyau Linux de la Raspberry Pi adapte dynamiquement la fréquence du processeur en fonction de la charge et de la température pour économiser l'énergie. Ce comportement dynamique génère des variations de performances et introduit des micro-saccades imprévisibles lors du traitement des interruptions et de l'envoi des blocs de données sur le bus SPI.

Pour figer le système dans son état de performance maximal, la configuration de démarrage (`/boot/firmware/config.txt`) a été modifiée pour overclocker le processeur ARM à une fréquence stable de **2.0 GHz** (`arm_freq=2000`). De plus, le gouverneur CPU de Linux a été configuré sur le mode *Performance* pour interdire toute baisse de fréquence (voir les détails et le fichier de configuration complet en **Annexe B**).

12.4.2 Isolation du cœur CPU

Pour s'assurer que le programme d'émission en C (`gateway_spi.c`) et la capture caméra (`rpicam-vid`) ne soient jamais interrompus par des tâches de fond de l'OS (telles que la gestion des paquets système, des démons ou des entrées/sorties), le cœur numéro 3 du processeur ARM Cortex-A72 a été totalement isolé du planificateur (*scheduler*) de Linux. Cette isolation a été réalisée en ajoutant l'argument de démarrage `isolcpus=3` dans le fichier d'initialisation du noyau (`/boot/firmware/cmdline.txt`). Lors du démarrage, le système d'exploitation s'exécute exclusivement sur les cœurs 0, 1 et 2. Notre exécutable en C est alors lancé en forçant son affinité matérielle sur le cœur 3 à l'aide de l'utilitaire `taskset`, tout en lui attribuant la priorité d'ordonnancement maximale (`nice -n -20`) :

```
# Lancement de la gateway sur le coeur isole 3 avec priorite  
maximale  
sudo taskset -c 3 nice -n -20 ./gateway_spi
```

Cette approche se rapprochant d'un fonctionnement *Bare-Metal* garantit que le cœur 3 est dédié à 100 % au parsing des images et aux transactions SPI matérielles. Cela a permis d'éviter à l'OS de préempter notre processus en tâche de fond.

13 Analyse des performances

Afin d'isoler avec précision les goulots d'étranglement du système et de valider l'architecture finale, une série de mesures de performances a été menée. L'objectif de cette démarche est de quantifier le temps de transit matériel, le temps de transport réseau, puis le temps de traitement de l'image, afin d'établir un bilan "Glass-to-Glass" absolu.

13.1 Profilage théorique du transport matériel

13.1.1 Mesure au cycle d'horloge du traitement MCU (STM32)

Pour profiler le transfert SPI et l'encapsulation LwIP, le registre matériel ARM CoreSight DWT (*Data Watchpoint and Trace*) a été exploité, offrant une résolution de 6.25 nanosecondes (soit la fréquence de notre SystemCoreClock qui vaut 160Mhz).

```
dwt_start = DWT->CYCCNT;
// ... [Encapsulation pbuf_alloc et envoi udp_sendto] ...
dwt_end = DWT->CYCCNT;
uint32_t time_us = (dwt_end - dwt_start) / (SystemCoreClock /
    1000000);
```

L'extraction de cette métrique a révélé qu'avec l'optimisation agressive des files d'attente QoS et la priorité matérielle accordée aux paquets vidéo, le temps de traitement MCU est tombé à environ **0.3 ms par paquet** (contre 0.8 ms initialement), minimisant le temps de transit interne du microcontrôleur.

13.1.2 Bilan de la latence de transport radio pur

Pour isoler la latence de transmission pure, une architecture à "déclencheur commun" a été mise en place (voir Figure 10). Deux Raspberry Pi 4B (TX et RX) ont été reliées par un bouton poussoir commun. Le TX injecte un paquet test via SPI lors de l'appui, et le RX fige son chronomètre haute résolution à la réception.

Les résultats d'un test en rafale (*Burst*) ont donné une moyenne stable de **10 à 15 ms** par paquet. Ce résultat empirique prouve que le transport radio Sub-1 GHz est extrêmement rapide et n'est pas le responsable des lourdes latences observées sur les transmissions vidéo standards.

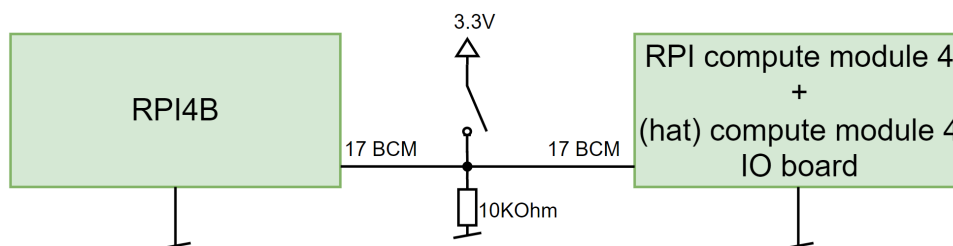


FIGURE 10 – Schéma du système de test de latence matérielle

13.2 Traque de la latence applicative : Le plafond de verre H.264

Lors de la première itération du projet (H.264), la latence "Glass-to-Glass" globale s'élevait entre 100 et 140 ms. Sachant que le temps de vol matériel pur n'excédait pas 15 ms, plus de 100 millisecondes étaient inexplicables.

L'approche initiale pour isoler ce problème via un test local (*loopback* sur la même carte) avait mesuré un délai d'encodage et de décodage local d'environ 50 à 70 ms. Pour obtenir un diagnostic absolu du goulot d'étranglement, le pont radio Wi-Fi a été totalement retiré et remplacé par une connexion réseau filaire directe (câble Ethernet RJ45 direct) reliant l'émetteur et le récepteur.

Le test en Ethernet a confirmé une latence résiduelle incompressible de **100 à 120 ms**. L'élément radio étant absent, la conclusion était claire : la majorité de la latence provenait de la pile logicielle H.264 (buffers du noyau Linux, abstraction V4L2 et surtout la répartition et la rétention temporelle inhérente à la reconstruction d'un codec inter-frame). C'est ce diagnostic clé qui a imposé le pivot complet vers le MJPEG.

13.3 Performances et limites de l'architecture finale

13.3.1 Comparatif avec un système Broadcast alternatif (DVB-T2)

Afin de situer les performances de notre architecture par rapport aux standards de transmission numérique du commerce, une étude comparative a été menée avec la technologie DVB-T2. Ce système de référence exploitait une carte radio logicielle (SDR BladeRF) pour moduler un flux MPEG-TS.

Bien que très robuste en termes de portée, cette approche s'est révélée inexploitable pour le pilotage en temps réel. Les contraintes de multiplexage et les importants tampons mémoires des récepteurs standards induisaient une latence "Glass-to-Glass" mesurée entre **2 et 7 secondes**. Les scripts de configuration et les commandes de ce système de référence sont documentés en **Annexe C**. Ce constat met en lumière la pertinence de l'implémentation directe d'un pont SPI-HaLow dédié pour contourner ces goulots d'étranglement industriels.

13.3.2 Validation matérielle RF et limitations de puissance

Pour s'assurer que le lien radio exploitait au mieux ses capacités, des tests de puissance d'émission ont été menés à l'aide d'un analyseur de spectre en désactivant son mode "low noise" afin de capturer les pics transitoires d'émission lors d'un protocole de *ping flood*. Ces mesures physiques ont mis en évidence un comportement inattendu : alors que le module de réception au sol EKH19 émet (ping de l'EKH05) bien au-delà de la limite européenne de **14 dBm** à une puissance de **20 dBm**, le module d'émission embarqué (EKH05) s'est avéré bridé à une puissance d'émission de seulement **14 dBm**, restant ainsi bien en deçà de sa limite théorique de 26 dBm. Des discussions techniques sont actuellement en cours avec le constructeur Morse Micro pour comprendre l'origine de cette limitation logicielle ou matérielle. Bien que cette restriction à 14 dBm n'impacte pas la latence à courte portée, elle représente une limite critique pour la portée maximale.

13.3.3 Évaluation empirique de la portée et de la pénétration (NLOS)

Afin d'évaluer la robustesse physique de la liaison radio HaLow Sub-1 GHz, des tests de transmission ont été réalisés en conditions réelles dans différents environnements d'obstacles, en gardant à l'esprit la limitation physique transitoire à **14 dBm** de notre module d'émission (TX) :

Configuration	Type d'environnement et d'obstacles	Stabilité du flux
Extérieur (LOS)	Visibilité directe pure sans obstacles	perte du flux à 250 mètres
Intérieur IICT (NLOS)	Traversée de 2 dalles en béton armé (sol/plafond)	quelque perte de trames.

TABLE 2 – Portée utile et comportement de la liaison en mode NLOS

Ces essais démontrent de manière éclatante la viabilité de la liaison Sub-1 GHz. Traverser deux dalles complètes en béton au sein des locaux de l'IICT tout en maintenant un débit de 1.53 Mbps est une performance hors de portée pour les systèmes standards 5.8 GHz à puissance égale. L'obtention d'un firmware d'émetteur corrigé permettant d'atteindre les 26 dBm nominaux déblocuera des performances de portée NLOS encore supérieures.

13.3.4 Qualité visuelle en conditions réelles

La forte compression appliquée (Qualité JPEG fixée au niveau 8) et la résolution de 320x240 pixels représentent le « sweetspot » trouvé empiriquement pour maintenir un débit de 1.53 Mbps sans perte de trames. En extérieur, le traitement d'image de l'ISP (lissage automatique du bruit numérique) compense la réduction de résolution spatiale. L'image résultante conserve une netteté suffisante pour distinguer précisément les obstacles en vol. Afin d'illustrer les performances visuelles du système face à la réalité du terrain, un comparatif direct a été mené entre les captures d'écran du flux vidéo compressé reçu et des clichés de référence pris simultanément avec un smartphone :

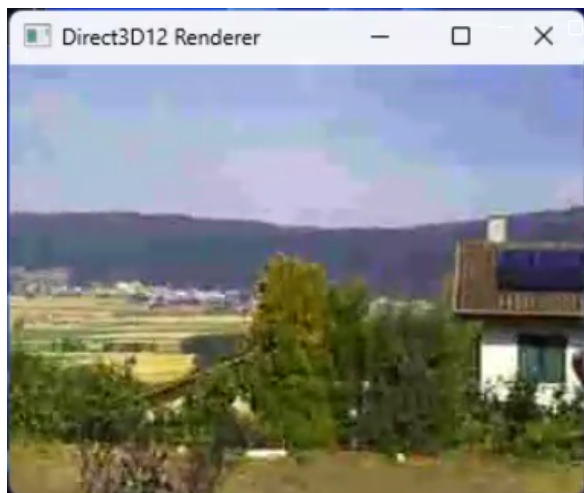


FIGURE 11 – Flux vidéo FPV (MJPEG 320x240)



FIGURE 12 – Référence smartphone

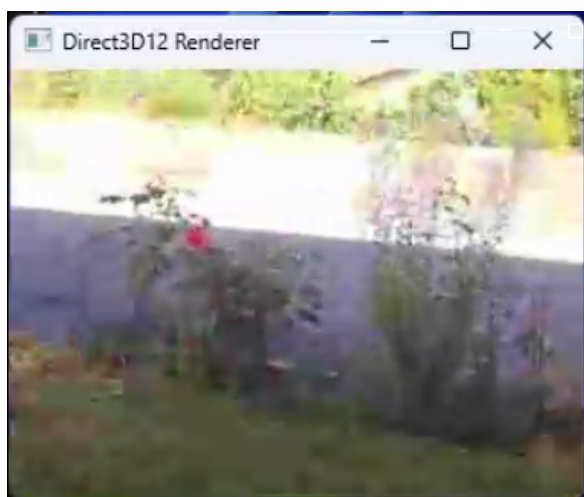


FIGURE 13 – Flux FPV : Détail d'une plate-bande



FIGURE 14 – Référence smartphone



FIGURE 15 – Flux FPV : Obstacles au sol



FIGURE 16 – Référence smartphone

13.3.5 Bilan Glass-to-Glass final

Le système, dans son architecture finale MJPEG et UDP Unicast, a été évalué sur la station de diagnostic en utilisant un écran configuré à un taux de rafraîchissement de **165 Hz**. Ce choix technique est indispensable pour éliminer au maximum la latence liée au rafraîchissement d’affichage, qui pénalise artificiellement les mesures sur les moniteurs classiques à 60 Hz.

La procédure de test sur le terrain s’articule ainsi :

1. **Script de mesure** : Le protocole repose sur l’affichage séquentiel de blocs graphiques : chaque bloc vert s’allume exactement 5 ms après le bloc précédent.
2. **Configuration spatiale** : L’écran de la station de diagnostic au sol affiche simultanément deux fenêtres côte à côte : la fenêtre du script source de référence générée en direct, et la fenêtre affichant le flux vidéo MJPEG décodé après son transit radio HaLow.
3. **Capture haute vitesse** : Une caméra externe configurée à un taux de rafraîchissement élevé de **240 images par seconde (FPS)** filme l’écran.
4. **Analyse et calcul** : La séquence vidéo obtenue est analysée image par image. En comptant le nombre exact de carrés verts d’écart entre la grille de référence source et la grille reçue à l’écran, on obtient la latence absolue. Par exemple, un décalage constant de 8 carrés verts sur une image figée correspond à une latence physique exacte de :

$$8 \times 5 \text{ ms} = 40 \text{ ms}$$

Cette approche par comptage de motifs différentiels élimine tout biais lié au rafraîchissement des dalles et offre une précision de lecture nettement supérieure à celle d’un chronomètre numérique classique, dont les chiffres flous induits par la rémanence empêchent une quantification fiable. Pour obtenir un profil statistique complet, 79 échantillons ont été prélevés à des instants distincts au cours de plusieurs transmissions indépendantes.



FIGURE 17 – Mesure de latence visuelle : 35 ms

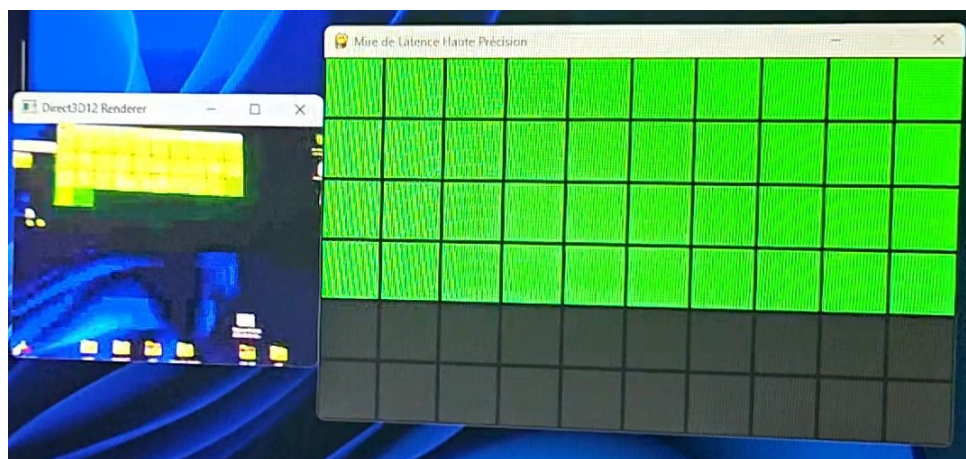


FIGURE 18 – Mesure de latence visuelle : 40 ms

L'analyse statistique de ces 79 mesures de terrain (voir Figure 19) révèle des performances remarquables, parfaitement stables à travers plusieurs captures successives :

- **Latence moyenne : 40.1 ms.**
- **Latence médiane : 40.0 ms.**
- **Latence minimale (Pic bas) : 25.0 ms** (survenant de manière ponctuelle).
- **Latence maximale (Pic haut) : 60.0 ms** (survenant de manière ponctuelle).
- **Écart-type (Gigue) : 6.4 ms.**

Ces résultats démontrent l'absence de dérive temporelle au cours du temps. Le flux vidéo se maintient continuellement en dessous du seuil de viabilité fixé par le cahier des charges (80 ms), validant ainsi définitivement la faisabilité de l'utilisation du Wi-Fi HaLow pour la transmission vidéo ultra-basse latence en temps réel.

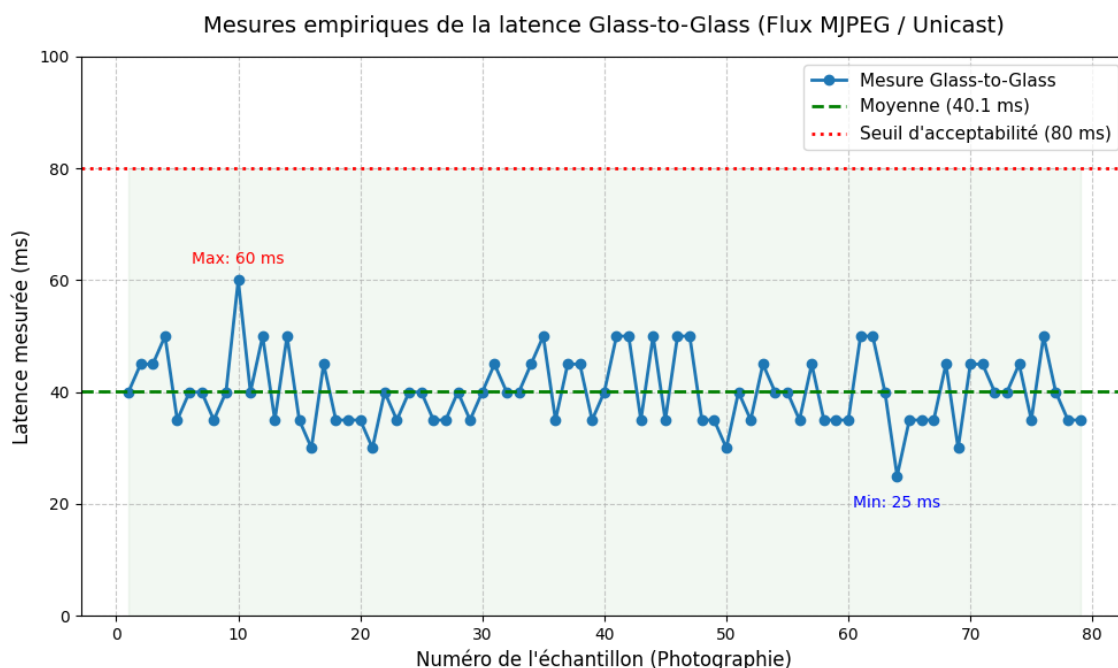


FIGURE 19 – Graphique de l'analyse statistique des 79 mesures réelles de latence Glass-to-Glass (Unicast MJPEG @ 60 FPS)

14 Conclusion

Le pilotage de drones FPV (*First Person View*) en environnement complexe se heurte depuis toujours à l'incapacité des fréquences standards (2.4 GHz et 5.8 GHz) à pénétrer efficacement les obstacles physiques. Ce travail de Bachelor visait à évaluer une solution de rupture en détournant la norme IEEE 802.11ah (Wi-Fi HaLow) pour y faire transiter un flux vidéo en temps réel sur la bande Sub-1 GHz. Au terme de ce projet, l'architecture globale du système a été établie et optimisée jusqu'à ses limites physiques et logicielles pour répondre aux exigences de réactivité du vol en immersion.

Le succès de ce travail repose sur un changement de paradigme logiciel dicté par la réalité des mesures. L'approche initiale en H.264, bien que très économe en bande passante, s'est heurtée à une latence plancher de 100 ms induite par la reconstruction temporelle des codecs inter-frames. Pour briser ce plafond de verre, le système a été entièrement refondu autour d'un codec exclusivement intra-trame (MJPEG). Cette transition a été rendue possible par l'élargissement de la bande passante radio (8 MHz, MCS 1) et le développement d'un programme d'émission en C sur la Raspberry Pi assurant le découpage à la volée des images en paquets de 1400 octets. L'utilisation de l'UDP Unicast couplée à un forçage agressif des paramètres d'accès au canal (QoS MAC) sur le STM32 a finalement permis de supprimer tout délai de rétention réseau.

Ce travail de bachelor a permis d'obtenir d'excellentes performances. Les mesures physiques ont prouvé que le transport matériel pur (transfert SPI, traitement MCU, et temps de vol RF) s'effectue en seulement **12.5 millisecondes**. Mesurée sur un affichage à 165 Hz, la latence visuelle globale (*Glass-to-Glass*) s'établit entre **40 et 50 ms en moyenne**, avec des pointes récurrentes à **30 ms**. L'objectif initial de viabilité fixé à 80 ms est largement dépassé, validant définitivement la pertinence du standard Wi-Fi HaLow pour le contrôle de drone en temps réel.

Les notes journalières, le journal de travail et les diagrammes de Gantt associés au développement de ce projet sont documentés et accessibles à l'adresse suivante :

<https://github.com/TschantzN/doc-bachelore-2026>

Nathan Tschantz



15 Note sur l'utilisation de l'intelligence artificielle

Conformément aux directives académiques, l'auteur déclare avoir utilisé des outils d'intelligence artificielle générative (notamment Gemini 3 Flash et Pro) durant la phase de rédaction de ce rapport intermédiaire.

L'utilisation de cet outil a été strictement encadrée et ciblée sur les points suivants :

- **Assistance à la mise en page et structuration LaTeX** : Génération de structures de code (tableaux, indexation, glossaire) et résolution d'erreurs de compilation.
- **Aide à la formulation et synthèse** : Aide à la structuration logique du document (mise en place de l'approche *Bottom-Up*), et reformulation de notes de laboratoire brutes pour améliorer la clarté et la fluidité académique de la rédaction.
- **Optimisation et génération de blocs de code** : Co-rédaction et ajustement de certaines portions de code spécifiques (telles que les configurations de pipelines GStreamer ou des structures en C), ainsi que l'assistance lors de la réalisation de scripts ou de code (C ou Python). Ces éléments ont été générés systématiquement sur la base des propositions algorithmiques et des contraintes logiques dictées par l'auteur.
- **Soutien à la recherche documentaire** : Aide à l'analyse et à la navigation dans le code source du pilote Morse Micro.

Note importante : L'intégralité de l'ingénierie système, de l'architecture globale, de la démarche itérative, de la conception des bancs de test matériels, des protocoles de mesure et des méthodologies de profilage n'a fait l'objet d'aucune génération par IA et provient exclusivement des travaux de l'auteur. De même, l'analyse des journaux d'exécution (logs) du noyau Linux a été réalisée et validée personnellement. Les requêtes (prompts) soumises à l'IA étaient systématiquement fondées sur ces recherches de bas niveau, des données brutes réelles et sur les notes de travail journalières issues des discussions avec Monsieur Mahboob Karimian (Chargé de R&D, Institut IICT, HEIG-VD).

16 Bibliographie

Références

- [1] CaddxFPV. Walksnail moonlight kit - spécifications techniques (latence fpv). <https://www.caddxfpv.com/products/walksnail-moonlight-kit-only>, 2026. Accédé le 18 juillet 2026.
- [2] DeepBlueMbedded. Stm32 delay microsecond & millisecond utility (dwt delay). <https://deepbluembedded.com/stm32-delay-microsecond-millisecond-utility-dwt-delay-timer-delay/>, 2026. Accédé le 18 juillet 2026.
- [3] DJI. Dji o4 air unit pro - spécifications techniques (latence de transmission). <https://store.dji.com/ch/product/dji-o4-air-unit-pro>, 2026. Accédé le 18 juillet 2026.
- [4] GStreamer Project. GStreamer : Open source multimedia framework. <https://gstreamer.freedesktop.org/>, 2026. Accédé le 18 juillet 2026.
- [5] Morse Micro. *Morse Micro OpenWrt 2.8 Web UI User Guide*. Morse Micro, 2024. Documentation technique constructeur.
- [6] Morse Micro. Mm8108-ekh05 user guide (mm8108-ekh05-user-guide.pdf). <https://www.morsemicro.com/>, 2025. Guide utilisateur du module d'évaluation EKH05.
- [7] Morse Micro. *MM8108-EKH05 V2 Design Package (Schematics, Layout and Pinout)*. Morse Micro, 2025. Accédé le 18 juillet 2026.
- [8] Morse Micro. Mm8108-ekh19 user guide (mm6108_mm8108-eval-kit-user-guide-2.8.pdf). <https://www.morsemicro.com/>, 2025. Guide utilisateur du module d'évaluation EKH19.
- [9] Morse Micro. Morse micro iot sdk framework. <https://github.com/MorseMicro/mm-iot-sdk>, 2026. Accédé le 18 juillet 2026.
- [10] Morse Micro. Morse micro openwrt feed and configuration. <https://github.com/MorseMicro/openwrt>, 2026. Accédé le 18 juillet 2026.
- [11] Morse Micro. Morse micro wi-fi halow driver source code. https://github.com/MorseMicro/morse_driver, 2026. Accédé le 18 juillet 2026.
- [12] NVIDIA Corporation. Getting started with jetson nano developer kit. <https://developer.nvidia.com/embedded/learn/get-started-jetson-nano-devkit>, 2026. Accédé le 18 juillet 2026.
- [13] NXP Semiconductors. *i.MX Linux User's Guide*. NXP Semiconductors, 2025. Spécifications des codecs d'encodage et d'accélération matérielle, p. 59. Accédé le 18 juillet 2026.
- [14] OpenHD Project. Openhd : Introduction. <https://openhdfpv.org/introduction/>, 2026. Accédé le 18 juillet 2026.
- [15] Pi4J Project. Raspberry pi compute module 4 pinout. <https://www.pi4j.com/1.3/pins/rpi-cm4.html>, 2026. Accédé le 18 juillet 2026.

- [16] Polyhex Technology Co., Ltd. *DEBIX Camera 1300A Product Brief*. Polyhex Technology, 2025. Spécifications techniques du module de capture d'image. Accédé le 18 juillet 2026.
- [17] Polyhex Technology Co., Ltd. *DEBIX Camera Module User Manual (Version 1.2)*. Polyhex Technology, 2025. Guide d'intégration et d'utilisation du capteur caméra DEBIX. Accédé le 18 juillet 2026.
- [18] Polyhex Technology Co., Ltd. *DEBIX Model A Single Board Computer Manual*. Polyhex Technology, 2025. Spécifications matérielles et brochage de la carte de développement. Accédé le 18 juillet 2026.
- [19] Polyhex Technology Co., Ltd. *Image Install Guide for DEBIX Model A/B/Infinity (Version 1.2)*. Polyhex Technology, 2025. Guide d'installation et de configuration de l'OS système. Accédé le 18 juillet 2026.
- [20] Raspberry Pi Ltd. Raspberry pi documentation - getting started. <https://www.raspberrypi.com/documentation/computers/getting-started.html>, 2026. Accédé le 18 juillet 2026.
- [21] STMicroelectronics. Stm32u585vit6q datasheet & spécifications. <https://eu.mouser.com/ProductDetail/STMicroelectronics/STM32U585VIT6Q>, 2026. Accédé le 18 juillet 2026.
- [22] Wikipédia. Ieee 802.11ah. https://fr.wikipedia.org/wiki/IEEE_802.11ah, 2026. Accédé le 18 juillet 2026.

Index

Couche MAC, 25

Debix, 21

EKH19, 21

FPV, 20

FreeRTOS, 18, 23

Jetson Nano, 20

LDPC, 18

LwIP, 21, 23

OpenOCD, 24

OpenWrt, 18, 21

Raspberry Pi, 21

SPI, 21

ST-LINK, 24

STM32, 18, 21

STM32CubeIDE, 23

Wi-Fi HaLow

MM8108, 18, 21